

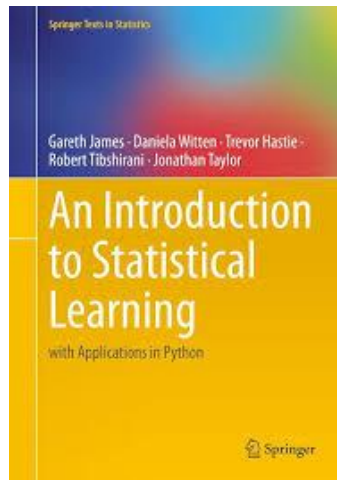
clusterAI

ciencia de datos en ingeniería industrial

UTN BA

curso I5521

clase_04: Clasificación

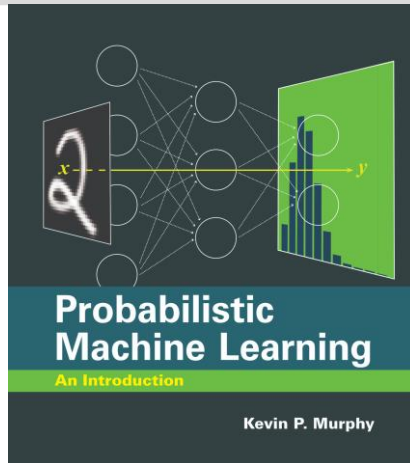


Introduction to Statistical Learning (with applications in Python)

<https://www.statlearning.com/>

https://hastie.su.domains/ISLP/ISLP_website.pdf.download.html

- Capítulo 1 -> completo
- Capítulo 2 -> página 15 a 34 (Introducción a Clasificación)
- Capítulo 4 -> página 135 a 144 (Regresión Logística)



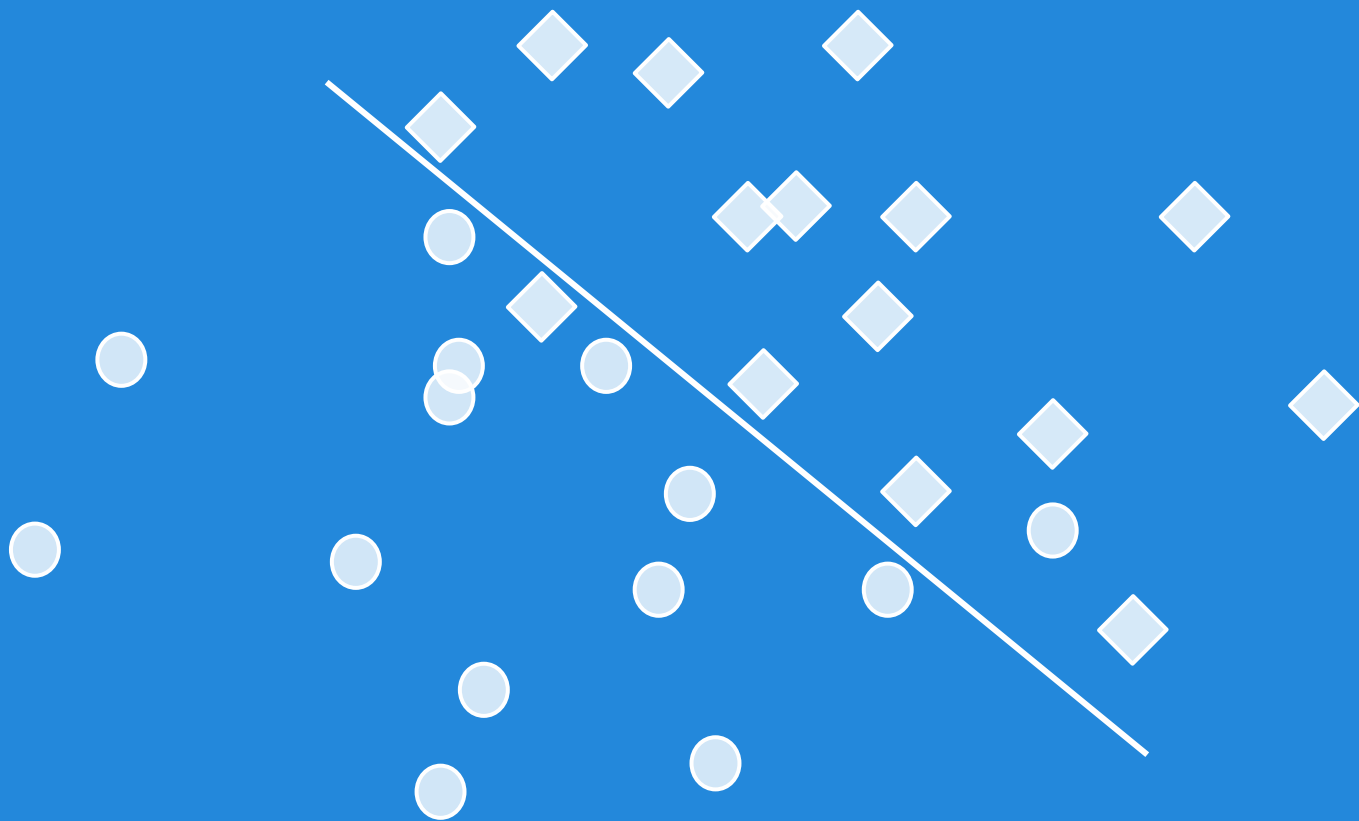
Probabilistic Machine Learning: An Introduction

<https://probml.github.io/pml-book/book1.html>

- Capítulo 1 -> Introducción
 - 1.1: What is machine learning?
 - 1.2: Supervised Learning
- Capítulo 10 -> Logistic Regression

agenda clase03: aprendizaje supervisado

- Clasificación
- Train-test split
- Cross validation
- Grid Search
- Performance metrics (Sens, Spec, ROC)
- Regularizacion
- Clasificadores: Regresión Logística



Aprendizaje supervisado: Clasificación

Aprendizaje supervisado

$$S = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$$

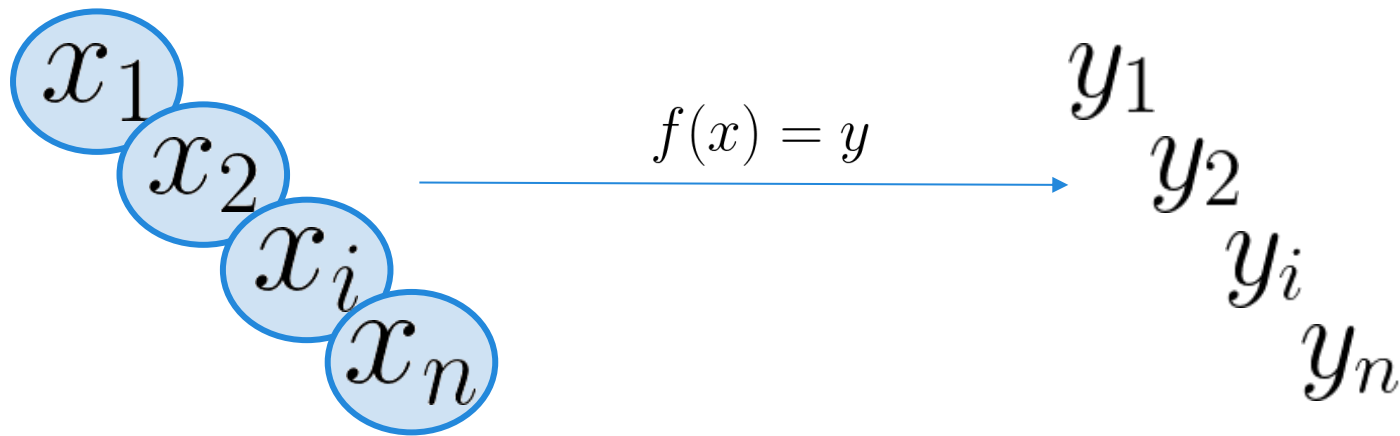
$$x \in \mathbb{R}^d \quad y \in \{-1, 1\} \quad f(x) = y$$

El enfoque de aprendizaje supervisado se basa en disponer datos ordenados en un dataset **S** en pares de instancias y etiquetas (samples 'x' & labels 'y'). Las instancias son vectores d-dimensionales de variables aleatorias i.i.d. (independientes e idénticamente distribuidos). Las etiquetas se suponen variables dependientes que pueden tomar valores discretos (clases) o continuos a partir de distintos valores de x mediante una función f(x) llamada 'ground truth' o función objetivo tal que f(x) = y. Es decir que f(.) explica la relación entre 'x' (input) e 'y' (output) Como la realidad es compleja generalmente no conocemos la verdadera f(x), por lo que trataremos de aproximarla.

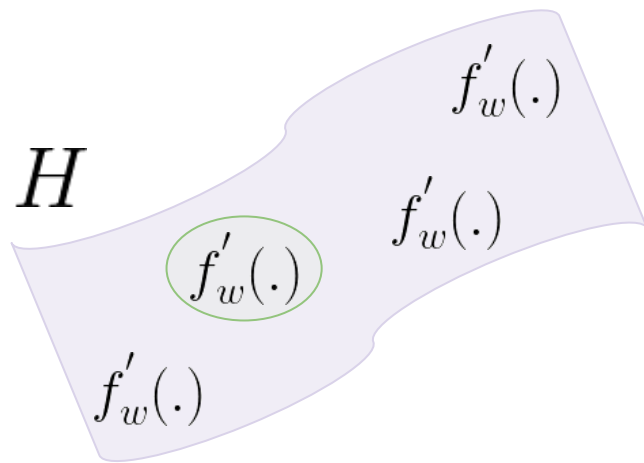
Aprendizaje supervisado

$$S = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$$

$$\mathcal{X} \in \mathbb{R}^d \quad y \in \{-1, 1\} \quad f(x) = y$$



Hipotesis



$$H = \left\{ f_w^1(\cdot), f_w^2(\cdot), \dots, \textcircled{f'_w(\cdot)}, \dots \right\}$$

Para aproximarnos a la verdadera $f(x)$ vamos a buscar alguna función $f(\cdot)$ dentro de un espacio de hipótesis que contiene muchas funciones $f(\cdot)$. De todas las funciones disponibles dentro del espacio de hipótesis H vamos a tratar de encontrar alguna que explique lo mejor posible la relación entre el input 'x' y el output 'y'. La función que vayamos a buscar estará caracterizada por **parámetros (w)** que pueden tomar distintos valores. Entonces existirá una combinación de parámetros que determinen una $f'(\cdot)$ que se aproxime a la verdadera $f(\cdot)$ mas que otras $f'(\cdot)$.

Aprendizaje supervisado

$$f(x) = y \quad \hat{f}(x) = \hat{y} \quad L(y, \hat{y})$$

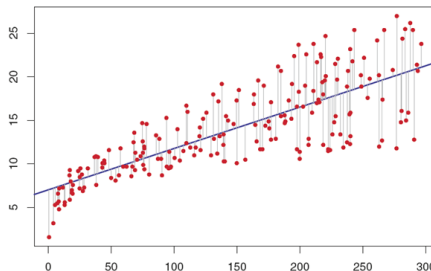
Suponiendo que tanto el dataset de sample-features y las etiquetas están disponibles $s=(x,y)$ vamos a **aprender** una función $f'(x)$ que explique lo mejor posible la relación (x,y) . Es decir aquella que aprenderemos una función que tomando como input las variables aleatorias “x” genere un output y' lo más similar a las etiquetas “y” dadas. Para poder medir cuán cerca están las etiquetas generadas por la función aprendida $f'(x)$ utilizaremos una función $L(y,y')$ de Costo o Pérdida (Loss function) que tomará valores altos cuando y sea muy distinto de y' . Por el contrario cuando y sea muy parecido a y' la función de costo tomará valores bajos. Por esta razón buscamos **minimizar** la función de costo.

$$L(y, \hat{y}) \qquad \hat{f}(x) = \hat{y}$$

El aprendizaje de los parámetros w para definir una función $f(x)$ durante la búsqueda de una frontera de decisión se realiza con métodos de optimización. Se busca minimizar la función objetivo de costo L utilizando las variables de decisión w , los parámetros de la función $f(x)$.

Métodos de aprendizaje supervisado

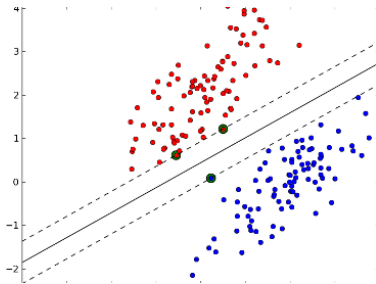
Regression



Y es continua

$$y \subseteq \mathbb{R}$$

Classification

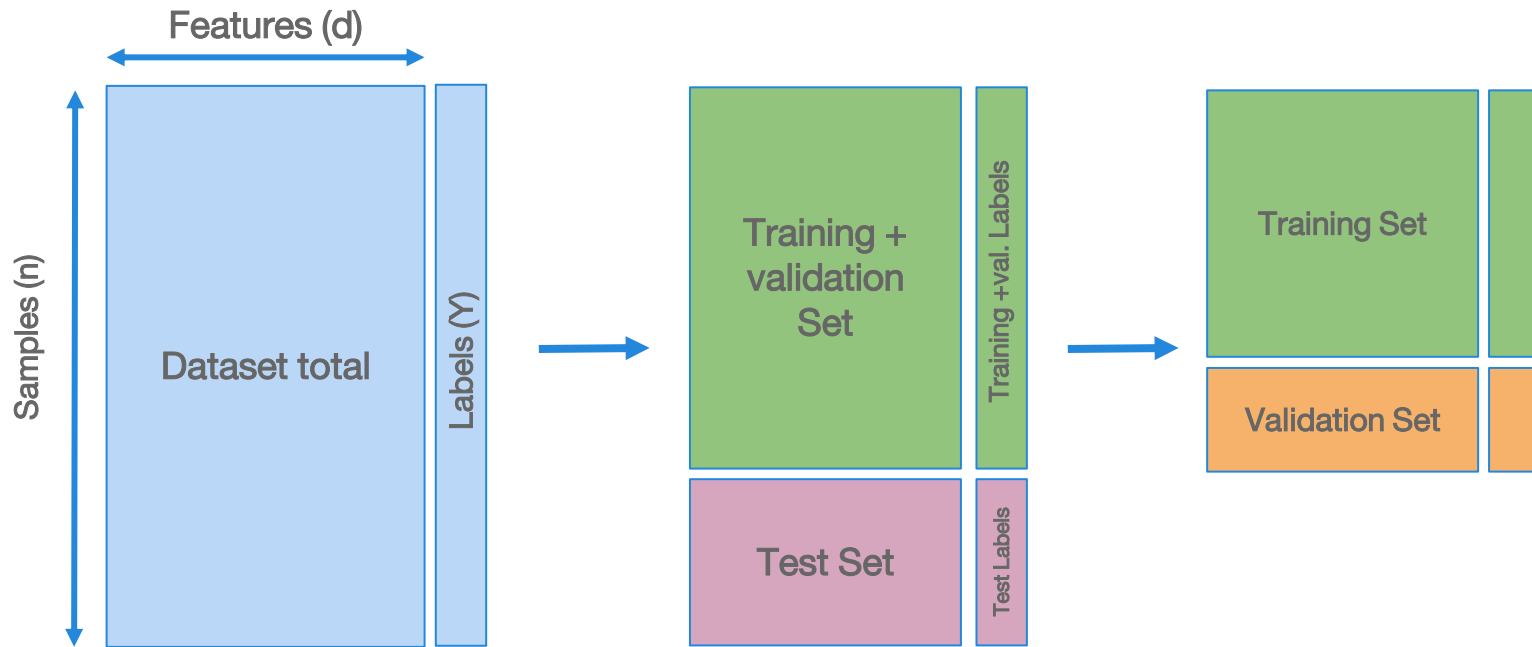


Y es categorica

$$y \in \{-1, 1\}$$

Existen dos enfoques importantes en el aprendizaje supervisado: clasificación y regresión. Cuando las etiquetas toman valores categóricos hablamos de clasificación. Cuando las etiquetas toman valores continuos hablamos de regresión.

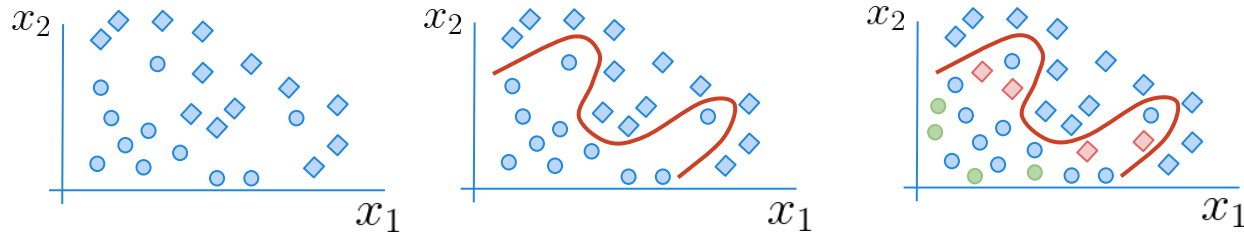
Train, Validation, Test sets.



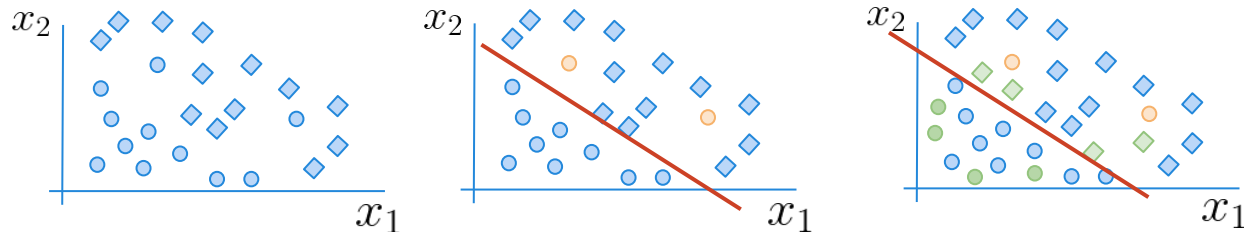
El clasificador aprenderá la regla de decisión utilizando el train set (samples + labels). Luego clasificará las muestras de test (sin mirar las labels de test) y se medirá la exactitud de clasificación en test.

Aprendizaje supervisado: clasificación

Modelo no-lineal



Modelo lineal



Datos de entrenamiento con etiquetas de dos clases.

Función de clasificación: entrenamos un modelo no-lineal y lineal.

Agregamos 4 muestras nuevas de cada clase y vemos cómo clasifican con el modelo entrenado.

Inicialmente el clasificador estará expuesto a muchas muestras de “entrenamiento” y sus respectivas etiquetas de manera tal de que aprenda los parámetros $\mathbf{w}$ que definen una función $f'(x)=y'$ cuya frontera **minimiza el error de clasificación $L(y,y')$** . Simultáneamente esperamos que luego de entrenar, la regla/función de decisión aprendida en los datos de entrenamiento clasifique bien muestras que nunca vió provenientes de la misma distribución/población (test). Es decir, que pueda **generalizar** a instancias ‘x’ nunca vistas clasificándolas correctamente.

En la figura: cuál modelo tiene menor error en el entrenamiento? cual modelo generaliza mejor? Cual es más complejo? Cual es más sencillo?

Frontera de decision lineal

Hiper-plano separador

$$f(x) = w^T x + b = 0$$

Funcion de decision

$$D(x) = \text{sign}[w^T x + b]$$

Existen muchos tipos de funciones de decisión. La familia de funciones más conocida es la de las funciones lineales. Estas funciones son hiper-planos caracterizados por parámetros $\mathbf{w}$ (vector $w=[w_1 \dots w_d]$) que determinarán cómo se posiciona la frontera de decisión en el hiper-espacio de dimensión d . En la clasificación binaria la función de decisión asignará un valor de $\mathbf{y=1}$ o $\mathbf{y=-1}$ según de que lado del hiper-plano se posicionen las muestras $\mathbf{x}$.

Variance vs Bias

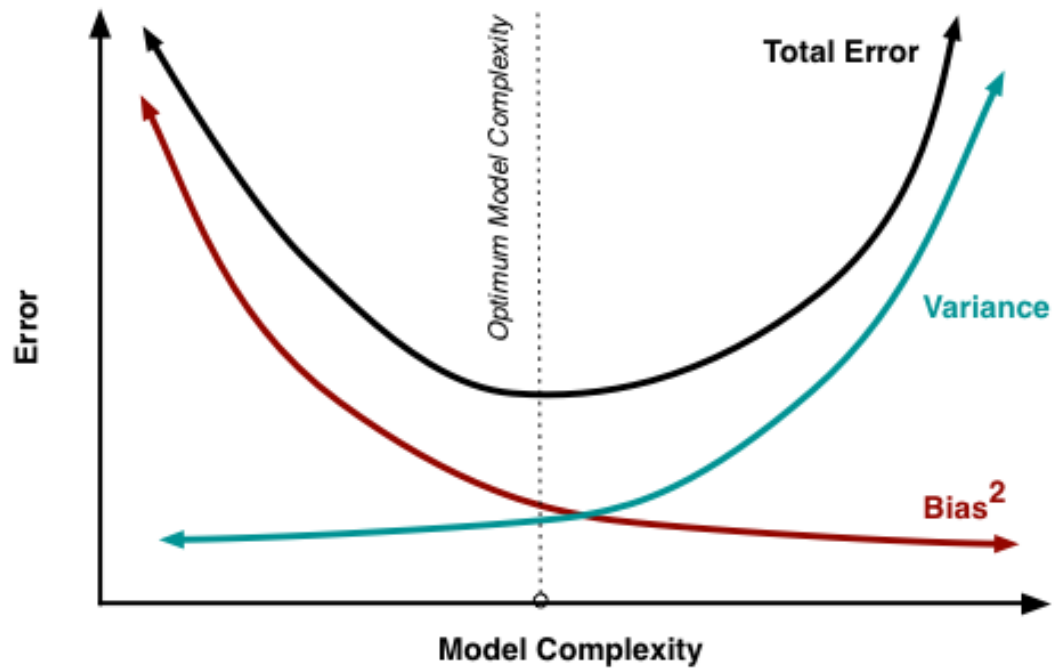
$$\text{Error} = \text{Var}(\hat{f}(x)) + [\text{bias}(\hat{f}(x))]^2$$

Cuando la etiqueta real $\mathbf{y}$ es distinta a la etiqueta asignada por el clasificador $\mathbf{y}'$ la función de pérdida-costo (Loss) $\mathbf{L}$ se incrementará. La causa de que un modelo supervisado tenga error en muestras nuevas se puede causar por su sesgo y/o por su varianza.

Un modelo tiene alta varianza cuando su predicción varía mucho si lo exponemos a muestras distintas provenientes de la misma población. En general la varianza se observa en modelos complejos (polinomios de alto grado), y está asociada al 'sobre-ajuste' (overfitting).

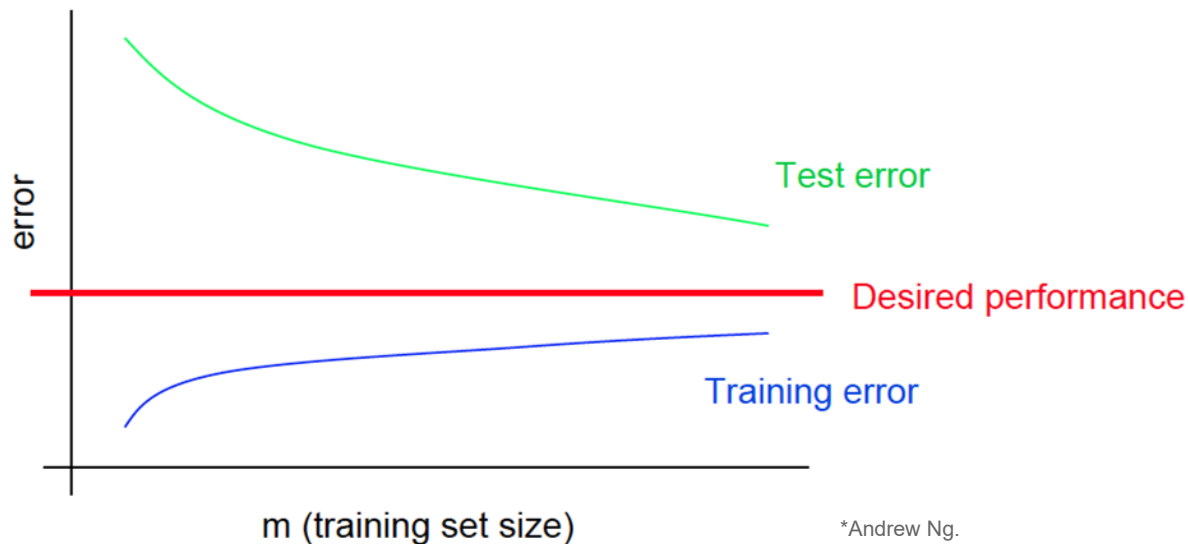
Los modelos con alto sesgo son aquellos que frente a distintos datos provenientes de la misma población no modifican mucho su predicción aunque en general ese valor de predicción no suele ser aceptable. El sesgo está asociado a 'sub-ajuste' (underfitting) y se suele observar en modelos muy sencillos (por ejemplo modelos lineales).

Variance vs Bias



*Andrew Ng.

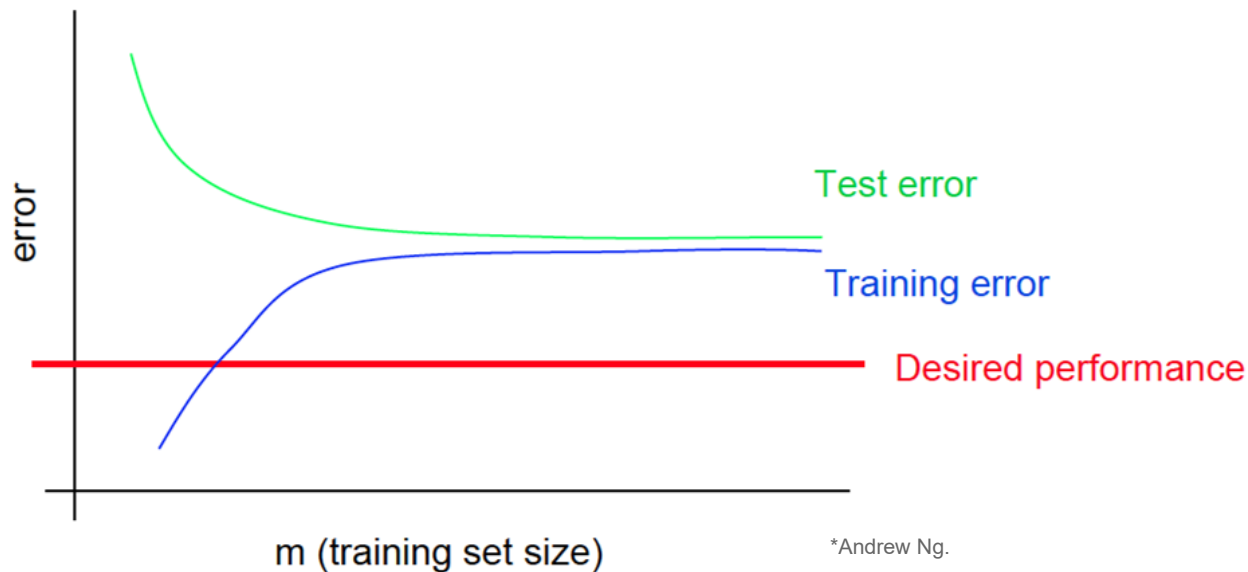
Learning curve: high variance



Higher model complexity might be affected by Variance

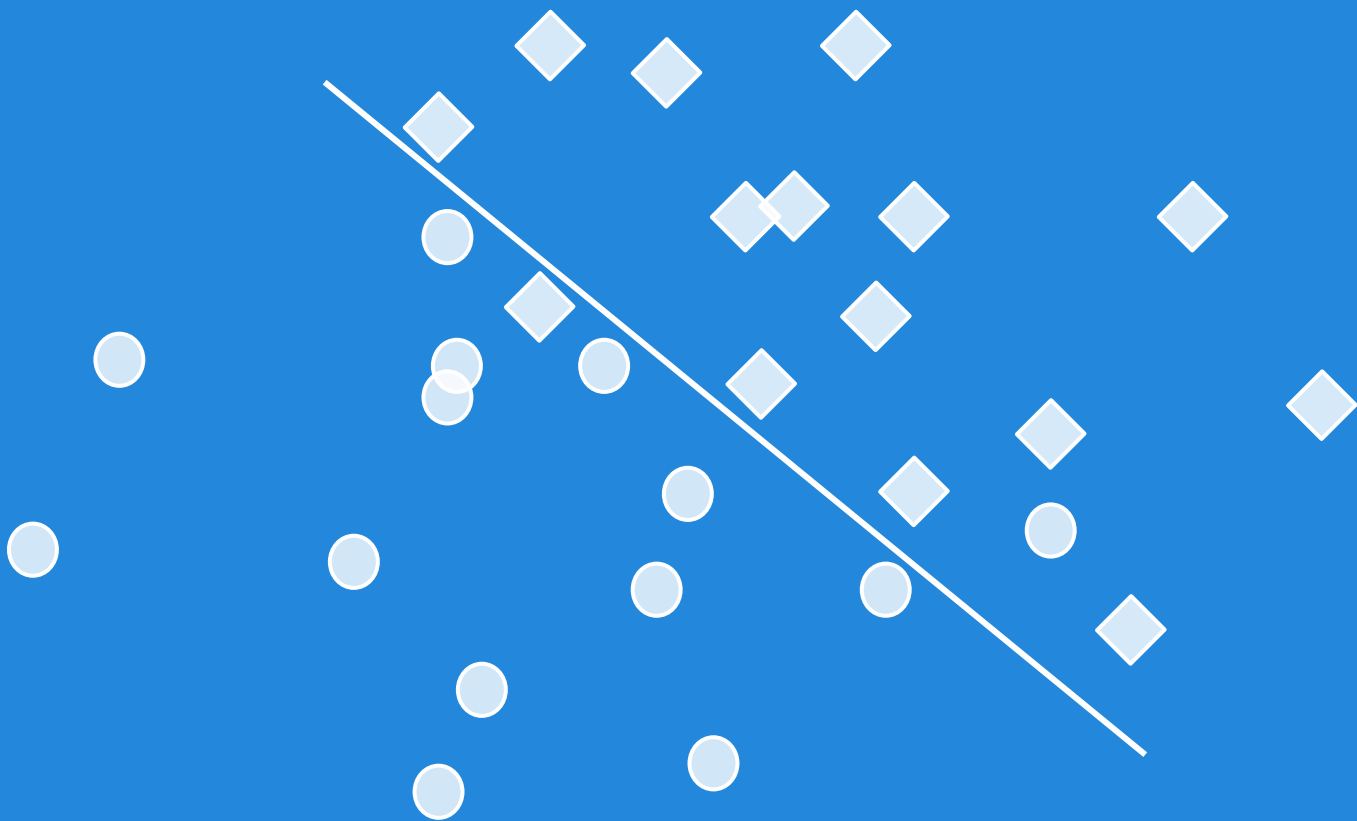
*Andrew Ng.

Learning curve: high bias



Simpler models might be exposed to Bias

*Andrew Ng.



Aprendizaje supervisado: mediciones de desempeño/performance en clasificación

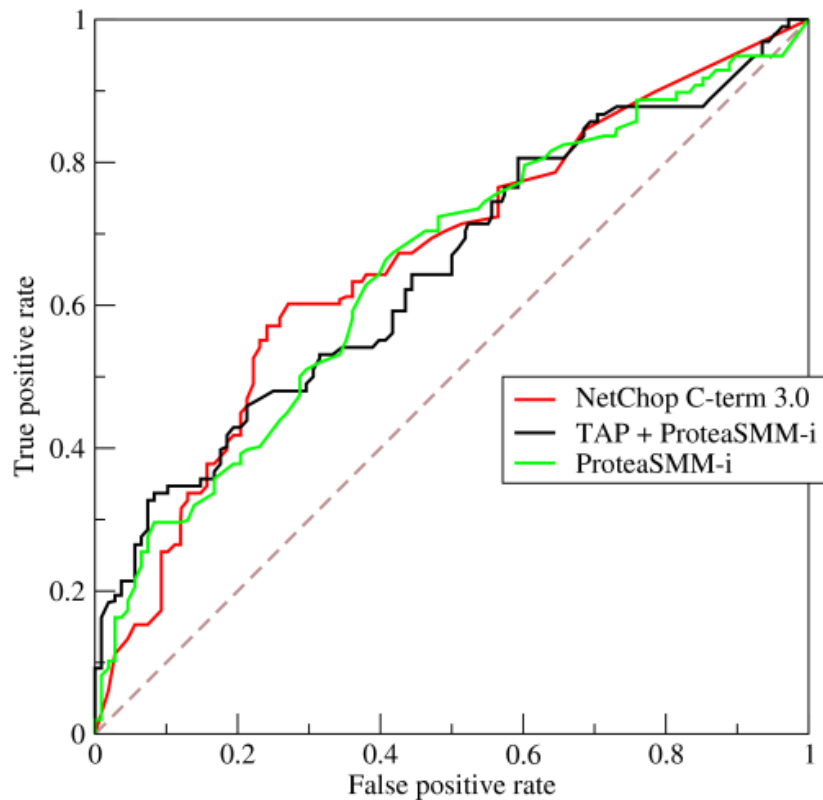
classification results: confusion matrix

		Predicted Label	
		Class1 (-)	Class2 (+)
True Label	Class1 (-)	True Negative	False Positive
	Class2 (+)	False Negative	True Positive

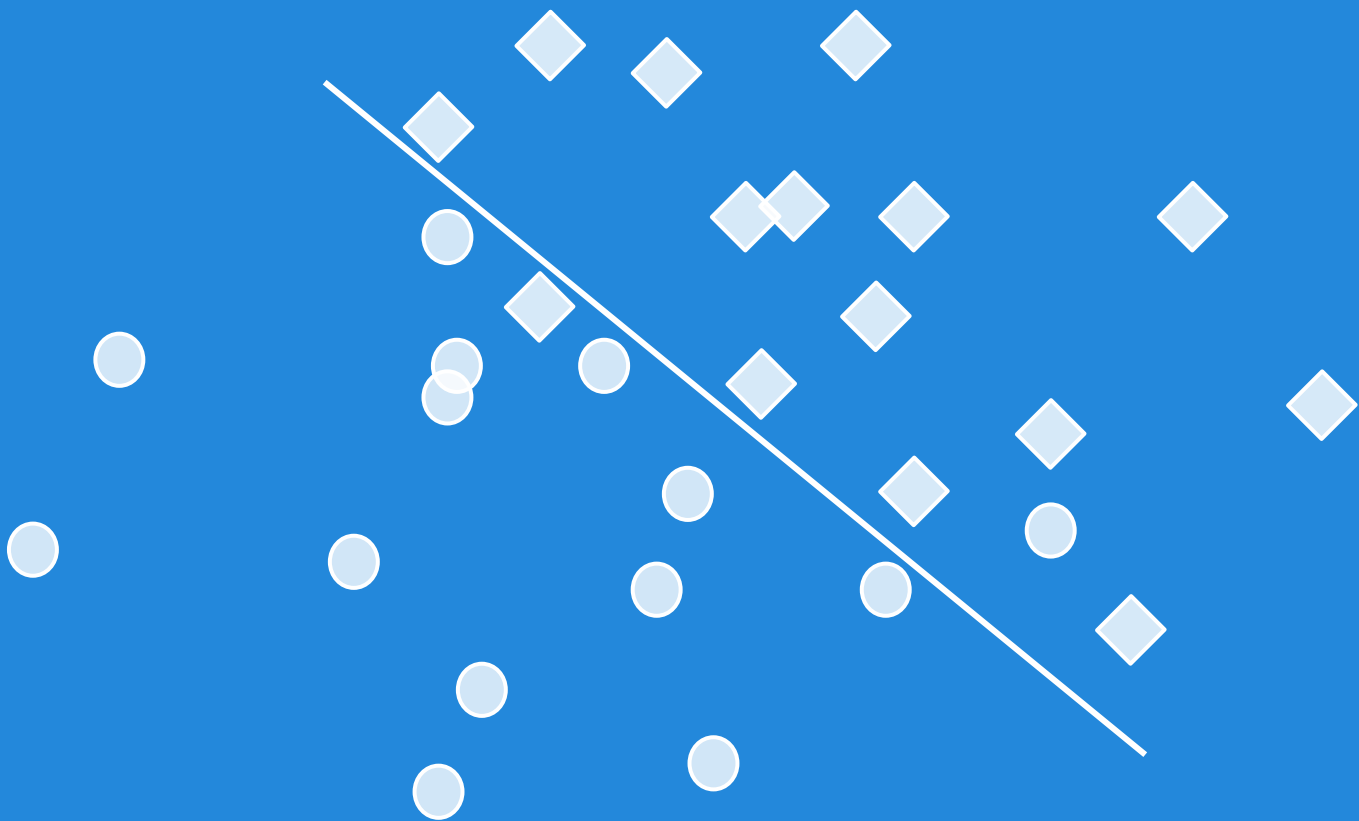
Accuracy = $(TN+TP) / \text{Total}$
Sensitivity (recall) = $TP/(TP+FN)$
Specificity = $TN/(TN+FP)$

La matriz de confusión es un elemento para evaluar los resultados de la clasificación. En cada posición se cuentan los TP, TN, FP, FN. Luego se obtienen los coeficientes de accuracy, Sensitivity, specificity.

classification results: AUC ROC



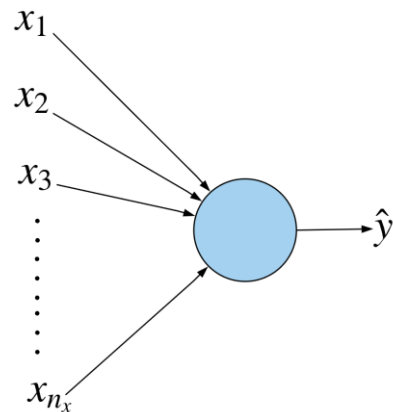
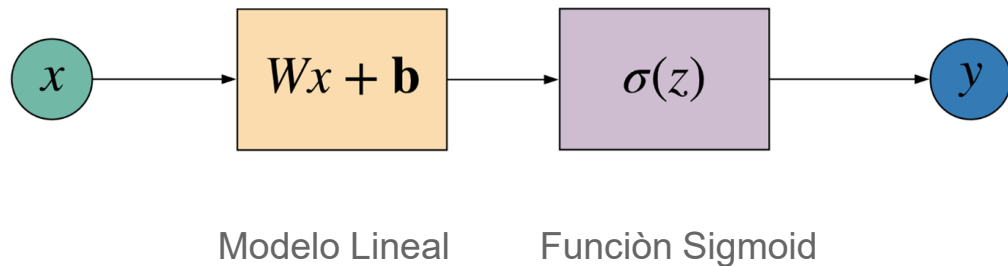
- El área bajo la curva ROC (AUC) da una idea de cuan bueno es mi clasificador independientemente del accuracy.
- Tomando una sola clase, se analizan distintos umbrales de clasificación y se contempla la relación entre TP y FP.
- Cuanto más cerca de 0.5 sea el AUC, más similar a “arrojar una moneda” será mi clasificador. Cuanto más cerca de 1 sea el área bajo la curva, mejor mi clasificador.



Aprendizaje supervisado, clasificación: Logistic Regression

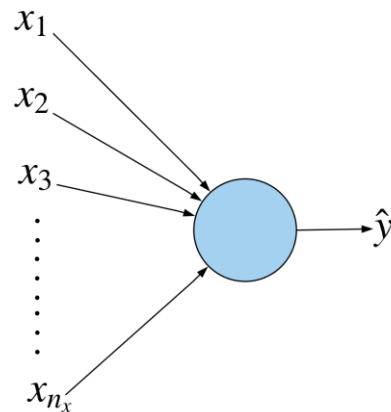
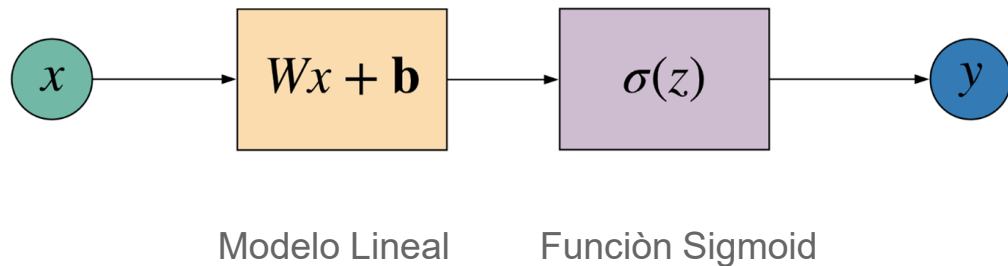
Logistic Regression

- Logistic regression es un clasificador lineal.
- Es una regresión lineal precedida de una función de activación “sigmoid”, lo que genera que el output sea binario y no continuo como una regresión normal.
- A cada muestra clasificada, le asigna una probabilidad de pertenecer a cada clase existente en el problema. Si la probabilidad es mayor a cierto threshold (0.5) entonces pertenece a una clase y viceversa.



Logistic Regression

- Logistic regression es un clasificador lineal.
- Es una regresión lineal precedida de una función de activación “sigmoid”, lo que genera que el output sea binario y no continuo como una regresión normal.
- A cada muestra clasificada, le asigna una probabilidad de pertenecer a cada clase existente en el problema. Si la probabilidad es mayor a cierto threshold (0.5) entonces pertenece a una clase y viceversa.



Logistic Regression

El regresor logístico debe aprender un parámetro interno (no es hiper-parámetro) por cada dimensión del vector de entrada (vector W). Para eso calculará el gradiente del error de clasificación y tratará de minimizarlo.

Probabilidad de la clase Y_i dado un vector de entrada x .

$$p(y_i|x) = \sigma(w^T x)$$

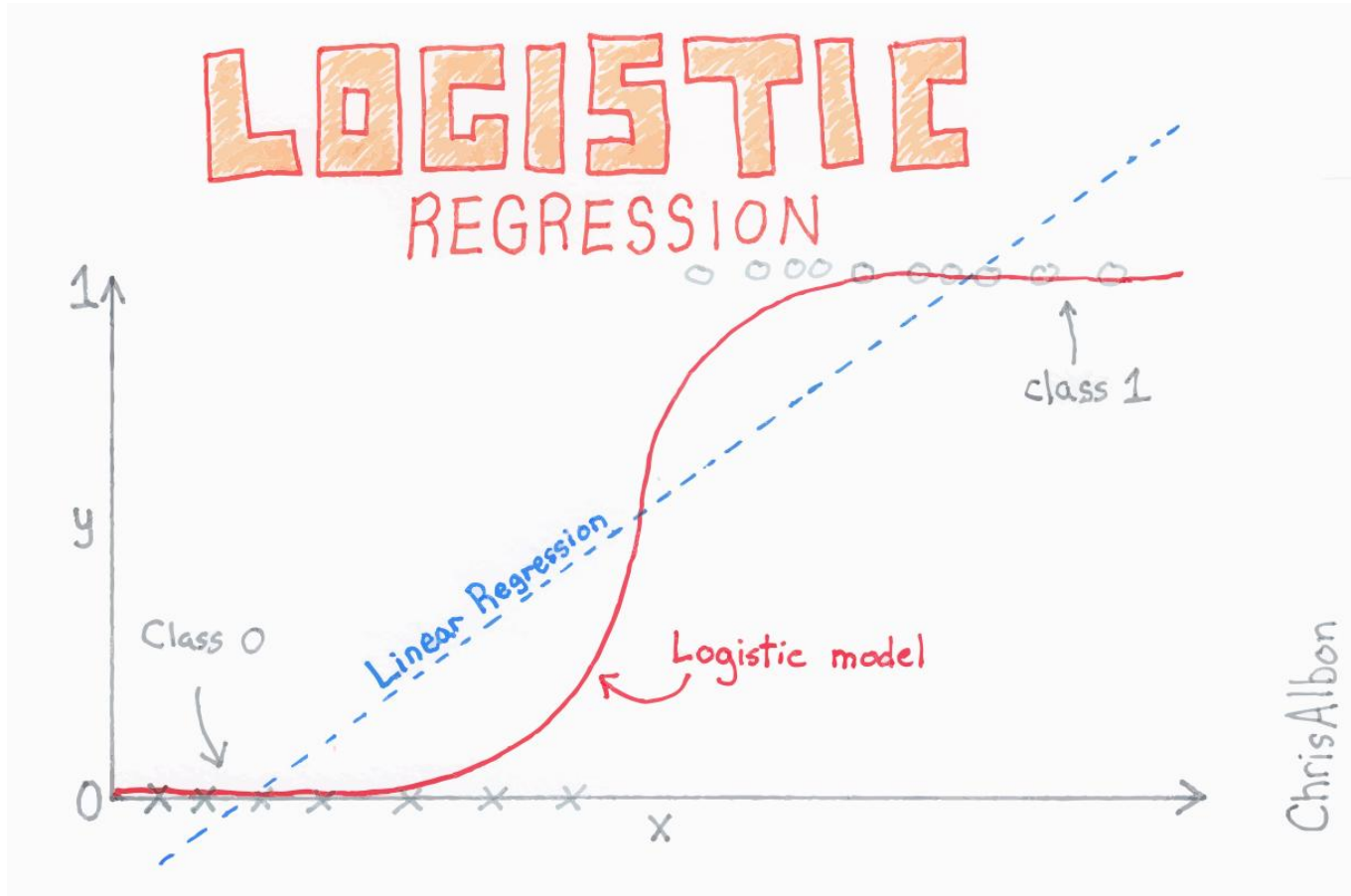
Función Sigmoid

La suma de las probabilidades de pertenecer a cada clase debe sumar 1.

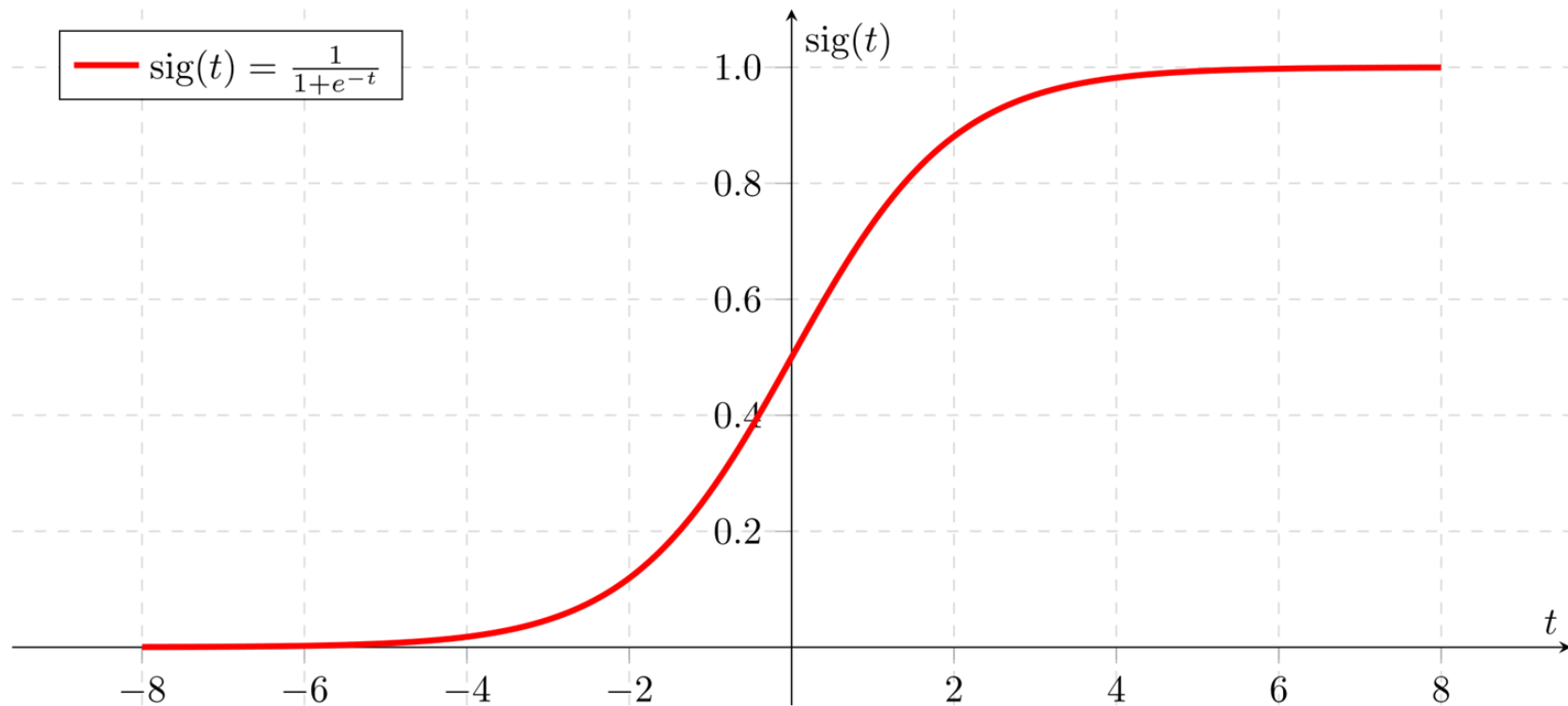
$$p(y_1|x) = 1 - p(y_2|x)$$

$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$

Logistic Regression



Logistic Regression



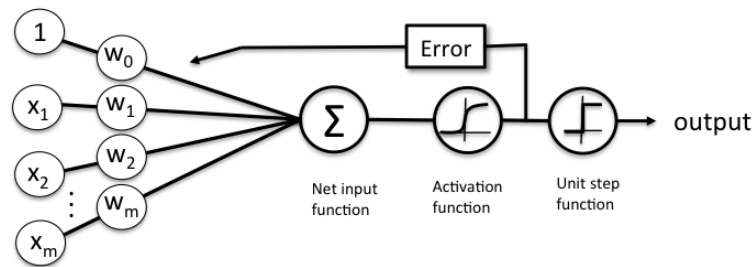
Logistic Regression

LOGISTIC VS. **LINEAR**
REGRESSION REGRESSION

$$p = \frac{1}{1 + e^{-(b_0 + b_1 x_1 + \dots + b_k x_k)}}$$

Logistic model Linear model

Chris Albon



Schematic of a logistic regression classifier.

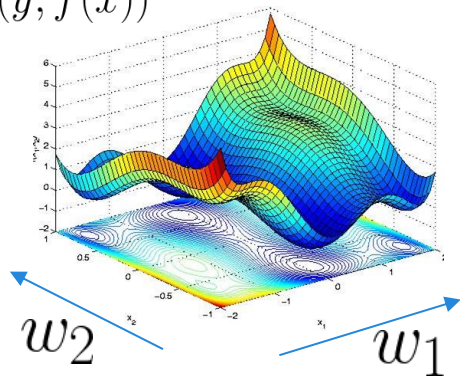
Lo que “aprende” logistic regression es el vector de “pesos” $\mathbf{W}$ de cada variable. Queremos que ese vector tenga “poder de generalización” para que clasifique bien nuevas muestras independientes una vez entrenado.

Logistic Regression: Optimizacion de funcion de costo

$$L_{\text{CE}}(\hat{y}, y) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$$

$$L_{\text{CE}}(\hat{y}, y) = -[y \log \sigma(w \cdot x + b) + (1 - y) \log(1 - \sigma(w \cdot x + b))]$$

$L(y, f(x))$

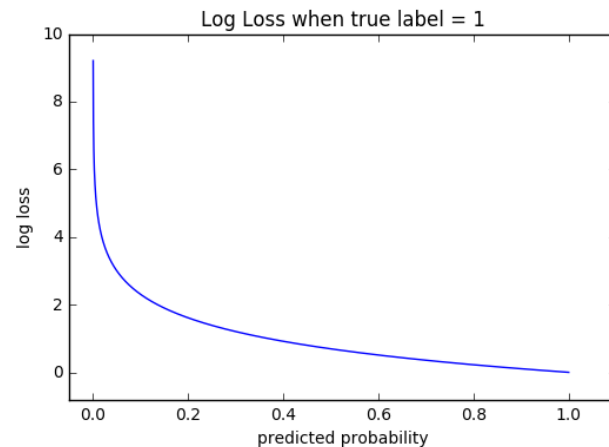


$$\hat{w} = \underset{w}{\operatorname{argmin}} \frac{1}{n} \sum_{i=1}^n L_{\text{CE}} \left(f \left(x^{(i)}; w \right), y^{(i)} \right)$$

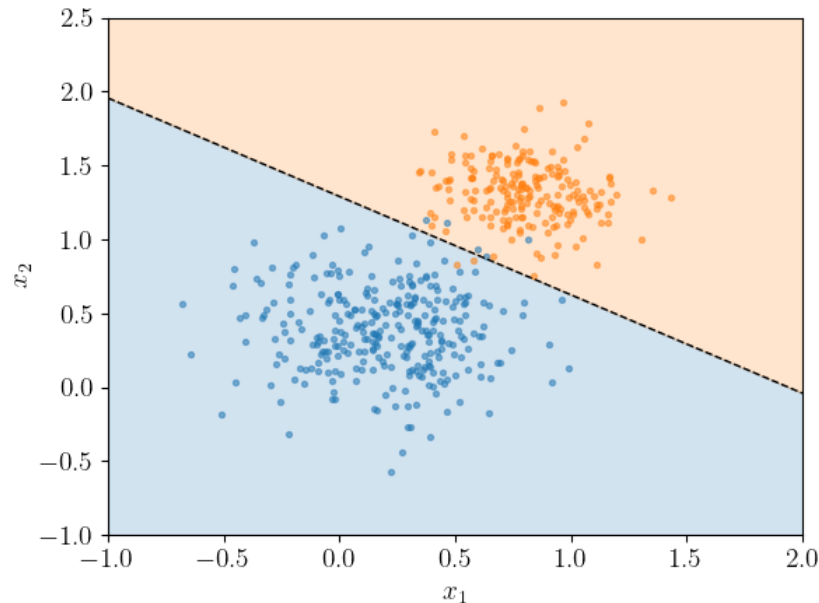
Para aprender la frontera de decisión con logistic regression se buscará encontrar los pesos w de la función lineal dentro de la función logística que minimicen el Loss Cross Entropy entre las etiquetas reales vs las etiquetas estimadas. La función de costo es no lineal y no convexa por lo que deberá utilizarse un optimizador de gradiente descendiente.

Logistic Regression Loss Function: Cross-Entropy

$$L_{\text{CE}}(\hat{y}, y) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$$



Logistic Regression: frontera de decision



La regresión logística es un método de clasificación lineal: busca separar las clases de forma tal de minimizar la función de costo (Cross Entropy). En el ejemplo hay 2 features/variables y 2 clases, sin embargo el método puede utilizarse en contextos de alta dimensión y múltiples clases.

Imagen: <https://scipython.com/blog/plotting-the-decision-boundary-of-a-logistic-regression-model/>

LR: Hiper Parametros

Los hiper-parámetros de Logistic Regression son

- **C = “Costo”** , es un penalizador que regulariza la solución computando clasificaciones erróneas.
- **Penalizador L1 o L2**: aplica una penalización bajo la norma L1 o L2 al vector **W**. De esta manera evita que el modelo quiera “sobre-ajustarse” a los datos de entrenamiento. En otras palabras, evita que existan posiciones de **W** muy muy altas y otras casi nulas, o viceversa, que solo algunas posiciones de **W** se activen y el resto no.

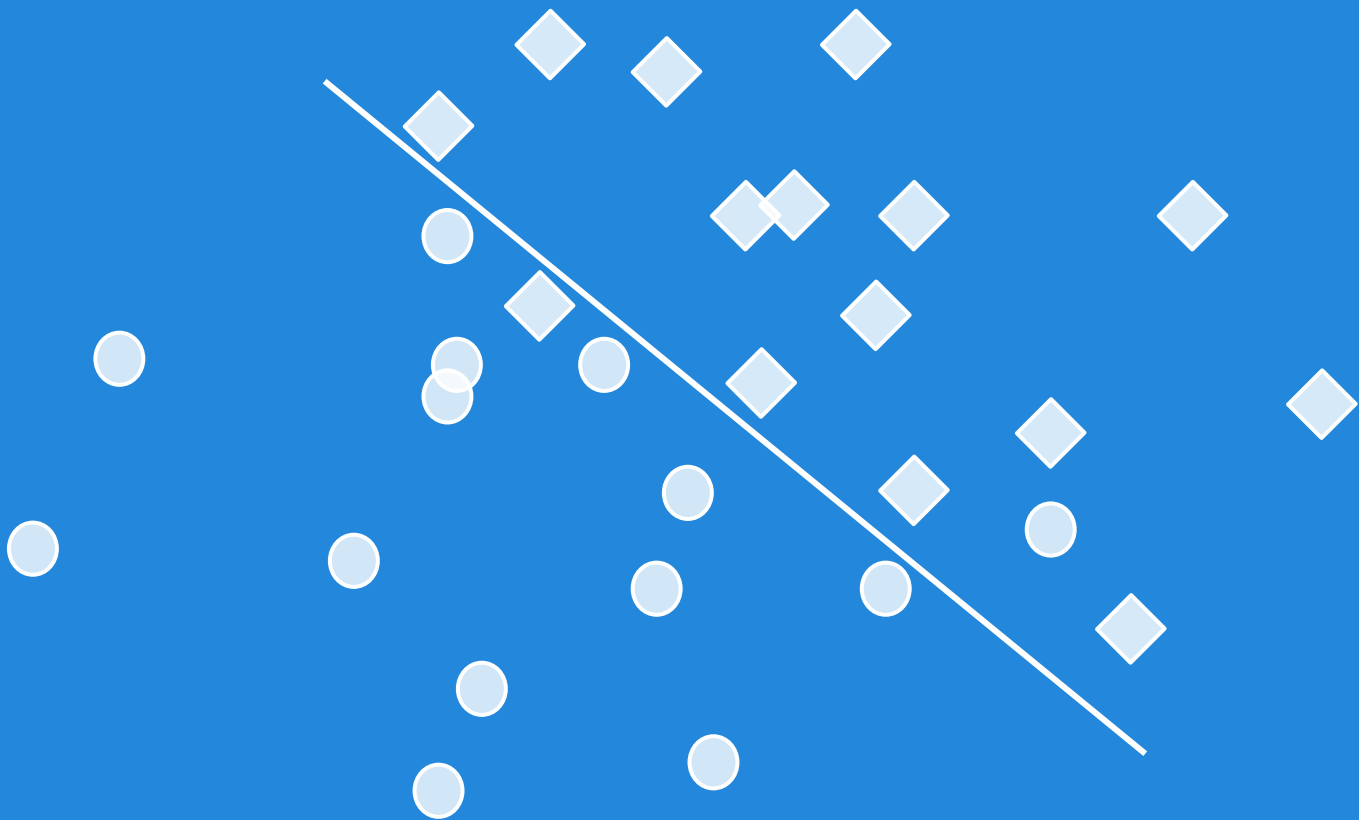
LR en Python con Scikit-Learn



```
# Samples and features
X = np.array([[-1, -1], [-2, -1], [1, 1], [2, 1]])

# Labels (two classes)
y = np.array([1, 1, 2, 2])

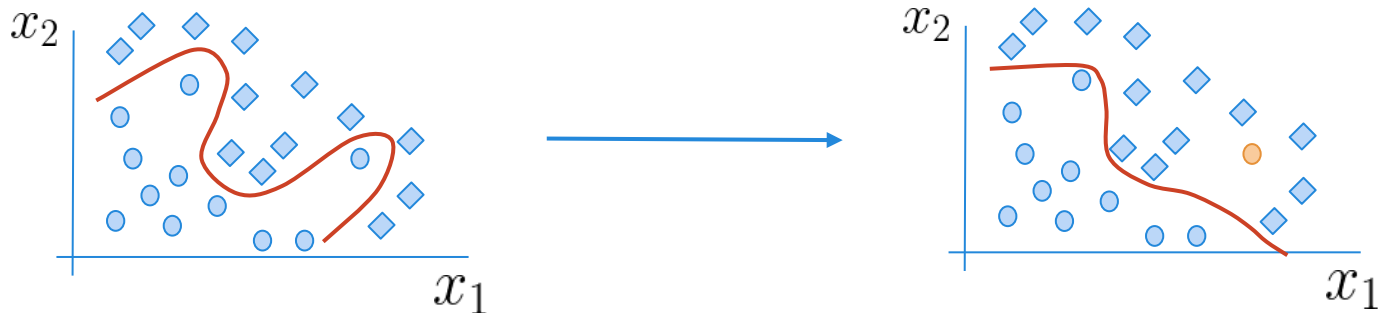
# import LR model
from sklearn.linear_model import LogisticRegression
# define KNN model
lr = LogisticRegression(random_state=0, solver='lbfgs')
# fit SVM on training data
lr.fit(X, y)
# predict a new independent sample
y_prediction = lr.predict([[-0.8, -1]])
# print prediction
print(y_prediction)
[0]
```



Aprendizaje supervisado: Regularizacion

Regularizacion

La regularización es un enfoque diseñado para mejorar la generalización de un modelo de aprendizaje supervisado y evitar el sobreajuste (overfitting).

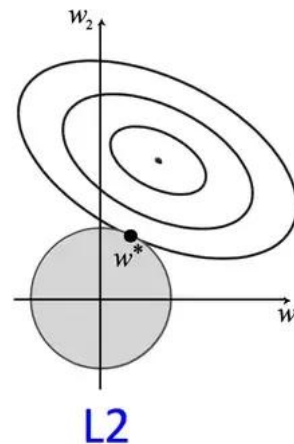
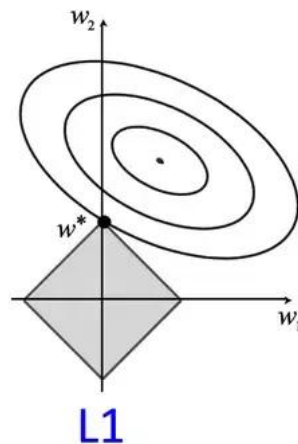


En muchos casos la regularización se asocia a **penalizar** al modelo por su complejidad obligándolo a no ajustarse completamente a los datos de entrenamiento. La regularización puede ser considerada un supuesto donde los datos ruidosos o muy complejos se asocian al ruido. Es decir que regularizar es suponer que la frontera de decisión no es tan compleja y esa suposición se la impone en el modelo como una penalización a la complejidad. Por el contrario una regularización excesiva puede generar un modelo demasiado simple que sub-ajuste a los datos. Por medio de la validación cruzada (cross validation) se tratará de encontrar un nivel de regularización aceptable.

Regularización de la función de decisión

$$\min_w L(y, \hat{y}) = \min_w L(y, \hat{f}(x))$$

$$f(x) = w_1 x_1 + w_2 x_2$$



Regularización con norma L2

$$\|\mathbf{w}\|_2 = (|w_1|^2 + |w_2|^2)^{1/2}$$

$$\|\mathbf{w}\|_2 < C$$

Regularización con norma L1

$$\|\mathbf{w}\|_1 = |w_1| + |w_2|$$

$$\|\mathbf{w}\|_1 < C$$

Regularización en LR: L1 & L2

Norma L1 : Lasso

$$\|\mathbf{w}\|_1 = |w_1| + |w_2| + \dots + |w_d|$$

Si le imponemos una restricción/penalización a la norma L1 de un vector lo que sucederá es que muchos valores de w tomarán el valor $w_i = 0$ para poder satisfacer la restricción y solo unos pocos serán distinto de cero.

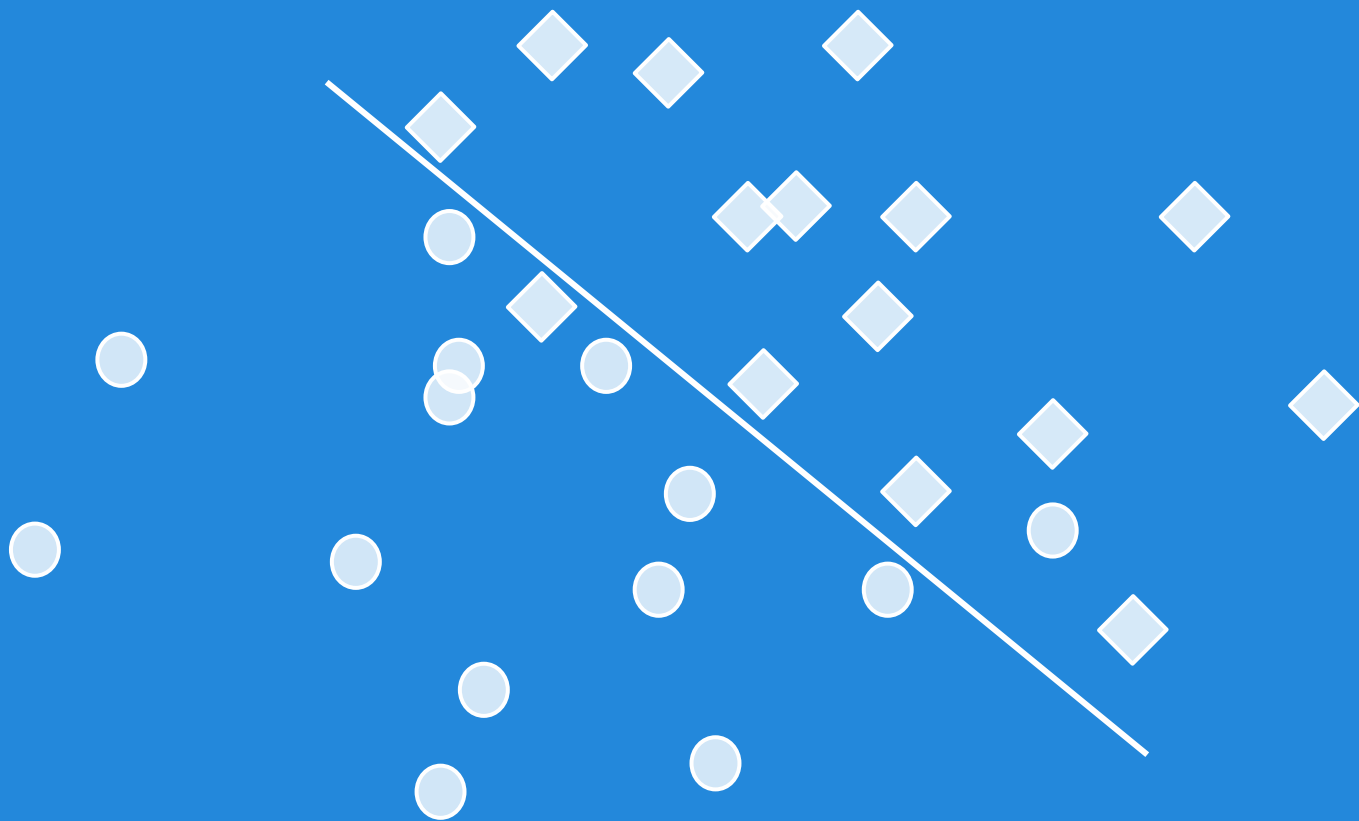
$$\|\mathbf{w}\|_1 < C$$

Norma L2 : Ridge

$$\|\mathbf{w}\|_2 = (|w_1|^2 + |w_2|^2 + \dots + |w_d|^2)^{1/2}$$

Si le imponemos una restricción/penalización a la norma L2 de un vector lo que sucederá es que muchos valores de w tomarán valores cercanos a cero para poder satisfacer la restricción y sólo unos pocos tendrán valores absolutos grandes.

$$\|\mathbf{w}\|_2 < C$$



Aprendizaje supervisado: Cross-Validation & Hyperparameter Tuning

Hiper-parámetros

$f_{w,\lambda}(x)$ funcion de decision
 w parametros
 λ hiper-parametros

Los modelos están caracterizados por parámetros $\mathbf{w}$ que son aprendidos durante el entrenamiento al ser expuestos a los datos. Adicionalmente los clasificadores tienen hiper-parámetros que definen entre otras cosas la familia de funciones que se pueden aprender. Por ejemplo, un hiper-parámetro podría ser el grado de una función polinomial. Los hiper-parámetros no son aprendidos por un algoritmo, son prefijados por el usuario. Los hiperparametros son útiles para poder determinar la complejidad y flexibilidad del clasificador. Por medio de una técnica llamada validación cruzada (cross validation) determinaremos cual la configuración de los hiper parámetros que minimiza el error de clasificación.

hyperparameters



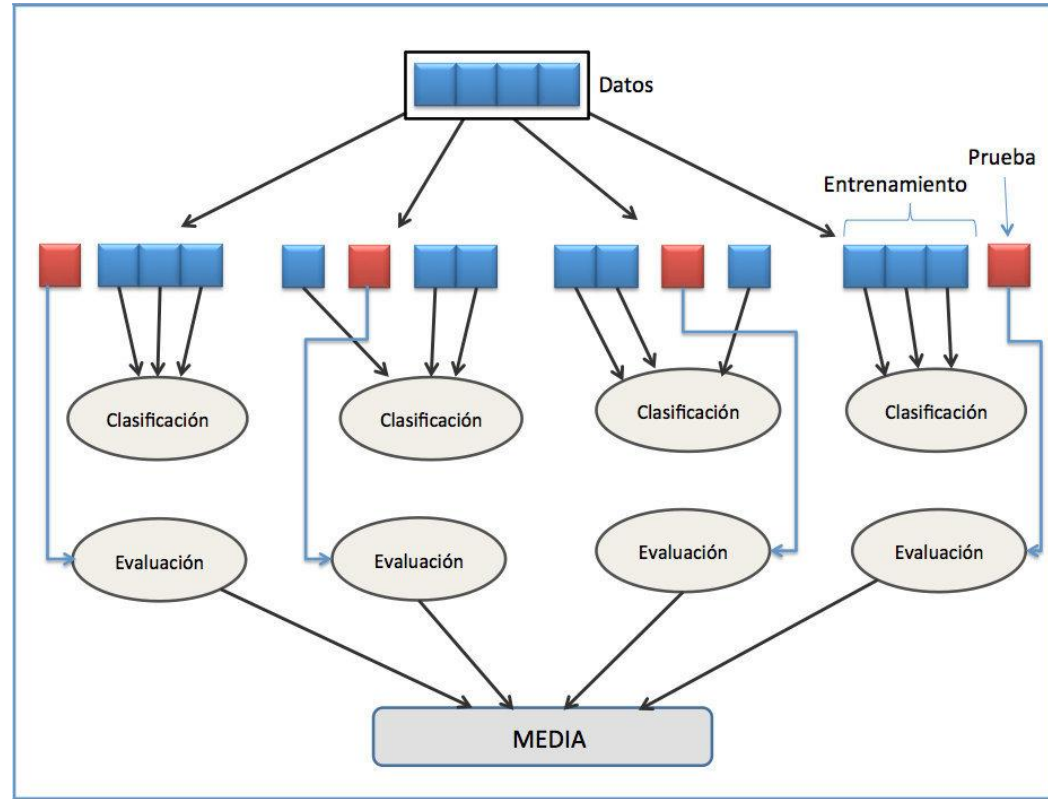
Los hiperparámetros son definidos por el usuario y no son determinados durante el aprendizaje interno del modelo. Para encontrar una combinación de hiperparámetros que permita un entrenamiento con resultados aceptables buscaremos en una grilla de valores de hiperparámetros. Nos quedaremos con la combinación de la grilla que menor media de error tenga luego de K iteraciones de validación cruzada (cross validation).

Cross - Validation en training set

“**Cross validation**” (CV) se realiza con las muestras de entrenamiento. Consiste en dividir nuestro training set en **K folds** (K porciones) e iterar K veces.

En cada iteración, una porción se utiliza como validación independiente y el resto como train. En cada iteración se entrena un modelo con train y se evaluará el resultado de clasificación con validación. Luego se realizará un promedio de la exactitud de clasificación de las k iteraciones.

Cross validation sirve para poder estimar el error estadísticamente lo que es equivalente tratar de entender cómo se comportaría frente a muestras nuevas. Además los valores de los hiperparámetros del modelo pueden determinarse por cross validation y se preserva el hiperparámetro que menor error promedio de cross validation genere.



Cross Validation

	$\delta_1 = 1$	$\delta_2 = 10$	$\delta_3 = 100$	$\delta_4 = 1000$	
$\lambda_1 = 0.01$	Acc_{11}^i	Acc_{12}^i	Acc_{13}^i	Acc_{14}^i	Fold = 0
$\lambda_2 = 0.1$	Acc_{21}^i	Acc_{22}^i	Acc_{23}^i	Acc_{24}^i	
$\lambda_3 = 1$	Acc_{31}^i	Acc_{32}^i	Acc_{33}^i	Acc_{34}^i	

	$\delta_1 = 1$	$\delta_2 = 10$	$\delta_3 = 100$	$\delta_4 = 1000$	
$\lambda_1 = 0.01$	Acc_{11}^i	Acc_{12}^i	Acc_{13}^i	Acc_{14}^i	Fold = 1
$\lambda_2 = 0.1$	Acc_{21}^i	Acc_{22}^i	Acc_{23}^i	Acc_{24}^i	
$\lambda_3 = 1$	Acc_{31}^i	Acc_{32}^i	Acc_{33}^i	Acc_{34}^i	

	$\delta_1 = 1$	$\delta_2 = 10$	$\delta_3 = 100$	$\delta_4 = 1000$	
$\lambda_1 = 0.01$	Acc_{11}^i	Acc_{12}^i	Acc_{13}^i	Acc_{14}^i	Fold = 2
$\lambda_2 = 0.1$	Acc_{21}^i	Acc_{22}^i	Acc_{23}^i	Acc_{24}^i	
$\lambda_3 = 1$	Acc_{31}^i	Acc_{32}^i	Acc_{33}^i	Acc_{34}^i	

	$\delta_1 = 1$	$\delta_2 = 10$	$\delta_3 = 100$	$\delta_4 = 1000$	
$\lambda_1 = 0.01$	Acc_{11}^i	Acc_{12}^i	Acc_{13}^i	Acc_{14}^i	Fold = 3
$\lambda_2 = 0.1$	Acc_{21}^i	Acc_{22}^i	Acc_{23}^i	Acc_{24}^i	
$\lambda_3 = 1$	Acc_{31}^i	Acc_{32}^i	Acc_{33}^i	Acc_{34}^i	

Supongamos que tenemos dos hiperparámetros y una grilla de valores para cada uno. Para cada combinación de hiper-parámetros se entrena un modelo y se calcula su exactitud (Accuracy). Luego se calcula la media de los resultados de cada una de las particiones de cross-validation y se selecciona la combinación que mejor resultado obtiene.

Pipeline: Train, Validate, Test Model

Dividir Train
y Test

Cross Validation &
Grid Search con Train
Set (utilizando Xtrain
e Ytrain)

Selección del
mejor modelo e
hiperparámetros

Clasificar muestras
de Test (Xtest) sin
mostrarle al modelo
las Ytest.

Evaluar
resultados de
clasificación en
test (comparar
Ypred vs Ytest)